When Hearts Collide

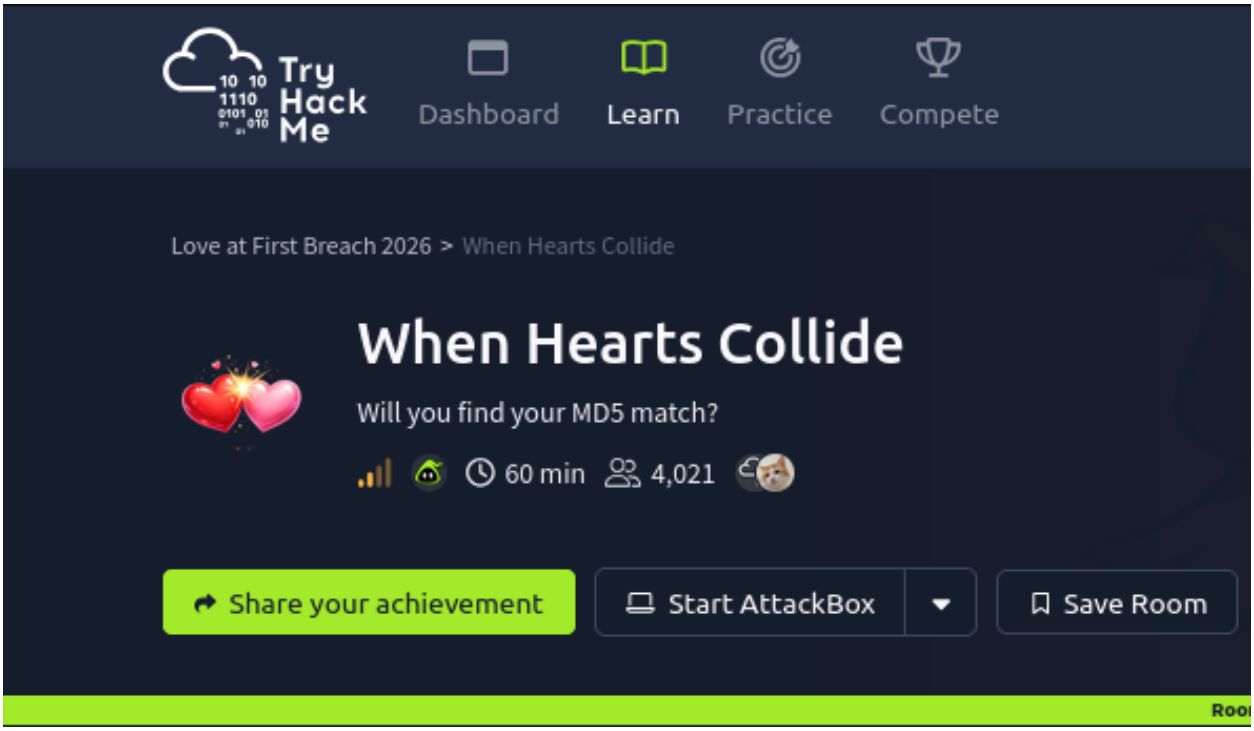
TryHackMe — Love at First Breach 2026

Lab Writeup / Student Report

Topic: MD5 Hash Collisions & Forging Matching Image Hashes

---

1. Lab Overview



Room Name	When Hearts Collide
Platform	TryHackMe (tryhackme.com/room/lafb2026e1)
Event	Love at First Breach 2026
Difficulty	Medium
Duration	60 minutes
Ports Found	22 (SSH) and 80 (HTTP)

Core Attack	MD5 Hash Collision using FastColl
Flag	THM{hash_puppies_4_all}

2. Objective

The challenge runs a web application that lets you upload a photo to find your 'matching dog'. Behind the scenes, when you upload an image, the app computes its MD5 hash and searches the database for a dog image that has the same hash.

The goal is to exploit a known weakness in MD5 — hash collisions — to upload two completely different image files that share the exact same MD5 hash. When you upload the second file, the app finds its match and returns the flag.

3. Background Knowledge

What is MD5?

MD5 is a hashing algorithm that converts any input into a fixed 32-character hexadecimal string. The same input always gives the same hash, and even a tiny change in the input produces a completely different hash — or so it should.

```
echo -n 'hello' | md5sum
5d41402abc4b2a76b9719d911017c592  -
```

What is FastColl?

FastColl is a tool that exploits the MD5 collision vulnerability. Given any input file, it generates two output files with different content but identical MD5 hashes — in seconds. This is the tool we used to forge our matching images.

4. Tools Used

- **network scanner, used to discover open ports on the target**nmap —
- **to interact with the dog-matching web application on port 80**Web browser —
- **MD5 collision generator; creates two files with the same MD5 hash**FastColl —
- **used to clone and compile the FastColl source code**git + g++ —
- **built-in Linux command to verify that two files share the same hash**md5sum —

- **to rename the generated collision files to .jpg so the web app accepts them**mv / cp —

5. Step-by-Step Solution

Scan the Target with Nmap

Start the machine on TryHackMe and note the IP address. Then run an nmap scan to find out which ports and services are running.

```
nmap -sC -sV 10.49.147.27
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 9.6p1
80/tcp open  http    nginx
```

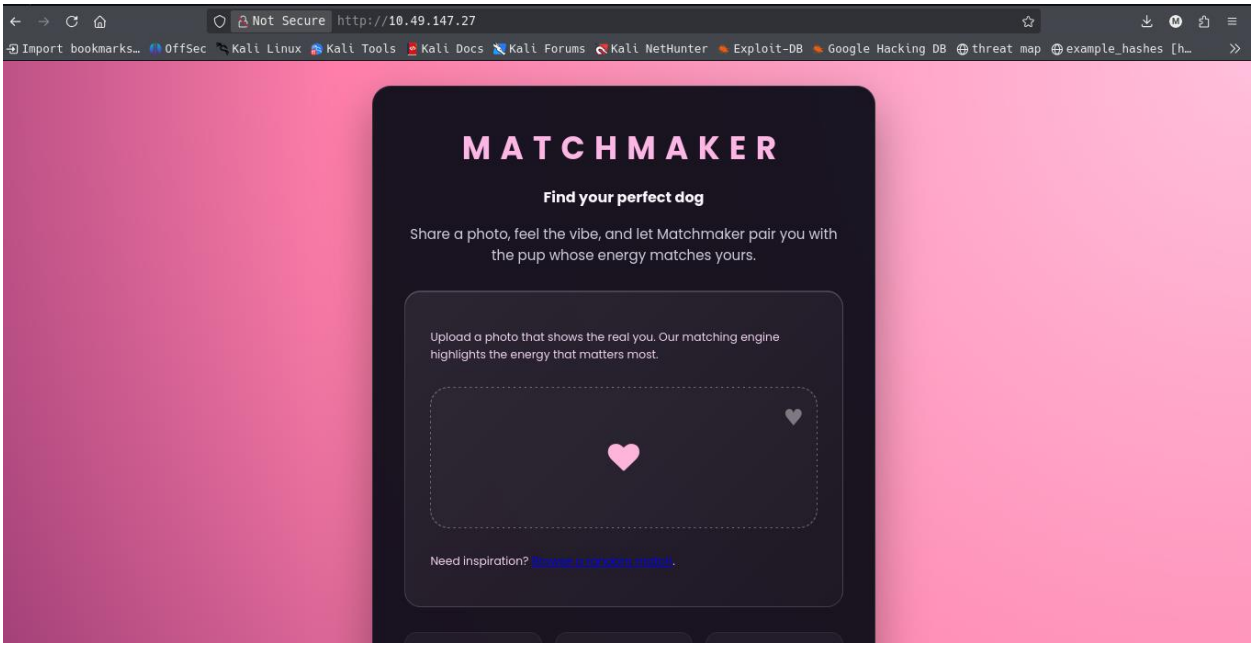
```
┌──(kn999㉿kn999)-[~]
└─$ nmap -sV -sC 10.49.147.27
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-19 17:35 +0300
Stats: 0:00:16 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 41.67% done; ETC: 17:35 (0:00:21 remaining)
Nmap scan report for 10.49.147.27
Host is up (0.38s latency).
Not shown: 998 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 9.6p1 Ubuntu 3ubuntu13.14 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 4c:8d:d3:ae:f4:fd:15:c5:d6:fc:20:65:36:a7:67:2f (ECDSA)
|_  256 f9:60:8f:1b:ae:93:dc:ae:5a:d7:b5:f7:3b:d0:72:4a (ED25519)
80/tcp open  http    nginx
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 157.71 seconds
```

Result: Only two ports are open — port 22 (SSH) and port 80 (HTTP web server). The room focuses on the web app, so we go to port 80.

Explore the Web Application

Open a browser and navigate to the machine's IP address on port 80:

```
http://10.49.147.27
```



The homepage of a dog-matching dating site appears. There are two options:

- Upload an image to find your matching dog
- Click 'Random Match' to see a sample match

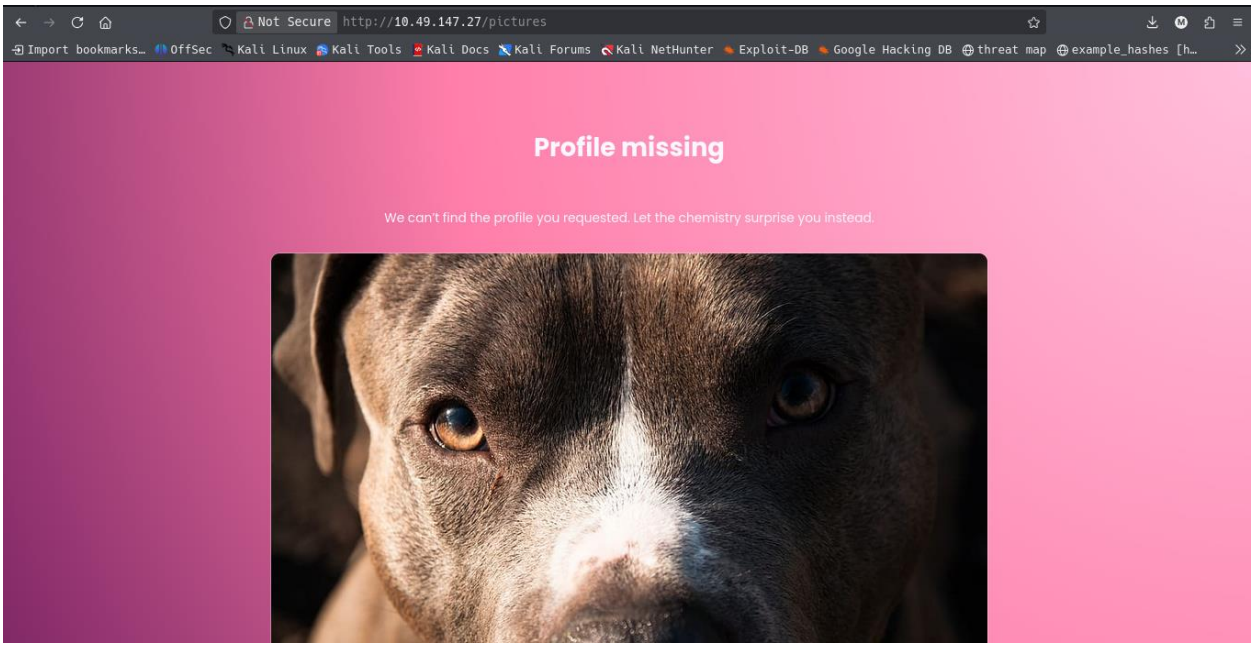
At the bottom of the page, a note explains: 'When you upload an image, its MD5 hash is computed and used to find a matching dog in our database.'

This tells us exactly what the app is doing — MD5 matching. Now we know the attack: if we can upload a file with the same MD5 as a dog already in the database, we get the match.

Download a Dog Image from the Site

To generate a valid collision, FastColl needs a real prefix file — in this case, an existing image from the website. Navigate to any dog image URL on the site, or just access a random path to get one:

```
http://10.149.47.27/pictures/
```



Download one of the dog images (e.g. thm-ab-bg-1803.png) to your local machine or AttackBox. Then move it to a working folder:

```
mkdir ~/Pictures
cd ~/Pictures
# Save the downloaded image here as: thm-ab-bg-1803.png
```

FastColl is not installed by default. We need to clone it from GitHub and compile it. Run the following commands:

```
git clone https://github.com/upbit/clone-fastcoll.git
```

```
┌──(root㉿kn999)-[/home/kn999/clone-fastcoll]
└─# git clone https://github.com/upbit/clone-fastcoll.git
Cloning into 'clone-fastcoll' ...
remote: Enumerating objects: 20, done.
remote: Total 20 (delta 0), reused 0 (delta 0), pack-reused 20 (from 1)
Receiving objects: 100% (20/20), 15.16 KiB | 75.00 KiB/s, done.
Resolving deltas: 100% (8/8), done.
```

After this, a compiled binary called 'fastcoll' is ready to use.

```
g++ -O3 *.cpp -lboost_filesystem -lboost_program_options -o fastcoll
```

```
┌──(kn999㉿kn999)-[~/clone-fastcoll]
└─$ g++ -O3 *.cpp -lboost_filesystem -lboost_program_options -o fastcoll
```

Generate Two Files with the Same MD5 Hash

Now run FastColl against the dog image we downloaded. This will produce two output files — twin_a.jpg and twin_b.jpg — that have completely different content but an identical MD5 hash:

```
cd ~/Pictures
../clone-fastcoll/fastcoll thm-ab-bg-1803.png
```

```
┌──(kn999㉿kn999)-[~/Pictures]
└─$ ../clone-fastcoll/fastcoll thm-ab-bg-1803.png
Generating first block: ....................
Generating second block: W..............
use 'md5sum md5_data*' check MD5
```

```
#This creates two files: md5_data1 and md5_data2
```

```
┌──(kn999㉿kn999)-[~/Pictures]
└─$ ls
md5_data1  md5_data2  thm-ab-bg-1803.png
```

Now verify that both output files have the same MD5 hash:

```
md5sum md5_data1 md5_data2
9f2afc5ecdf93e22f6e92014dbb78169  md5_data1
9f2afc5ecdf93e22f6e92014dbb78169  md5_data2
```

```
┌──(kn999㉿kn999)-[~/Pictures]
└─$ md5sum md5_data1 md5_data2
9f2afc5ecdf93e22f6e92014dbb78169  md5_data1
9f2afc5ecdf93e22f6e92014dbb78169  md5_data2
```

Both hashes match — the collision was successful. Now rename the files to .jpg so the web app accepts them as images:

```
mv md5s_data1 dog1.jpg
mv md5s_data2 dog2.jpg
```

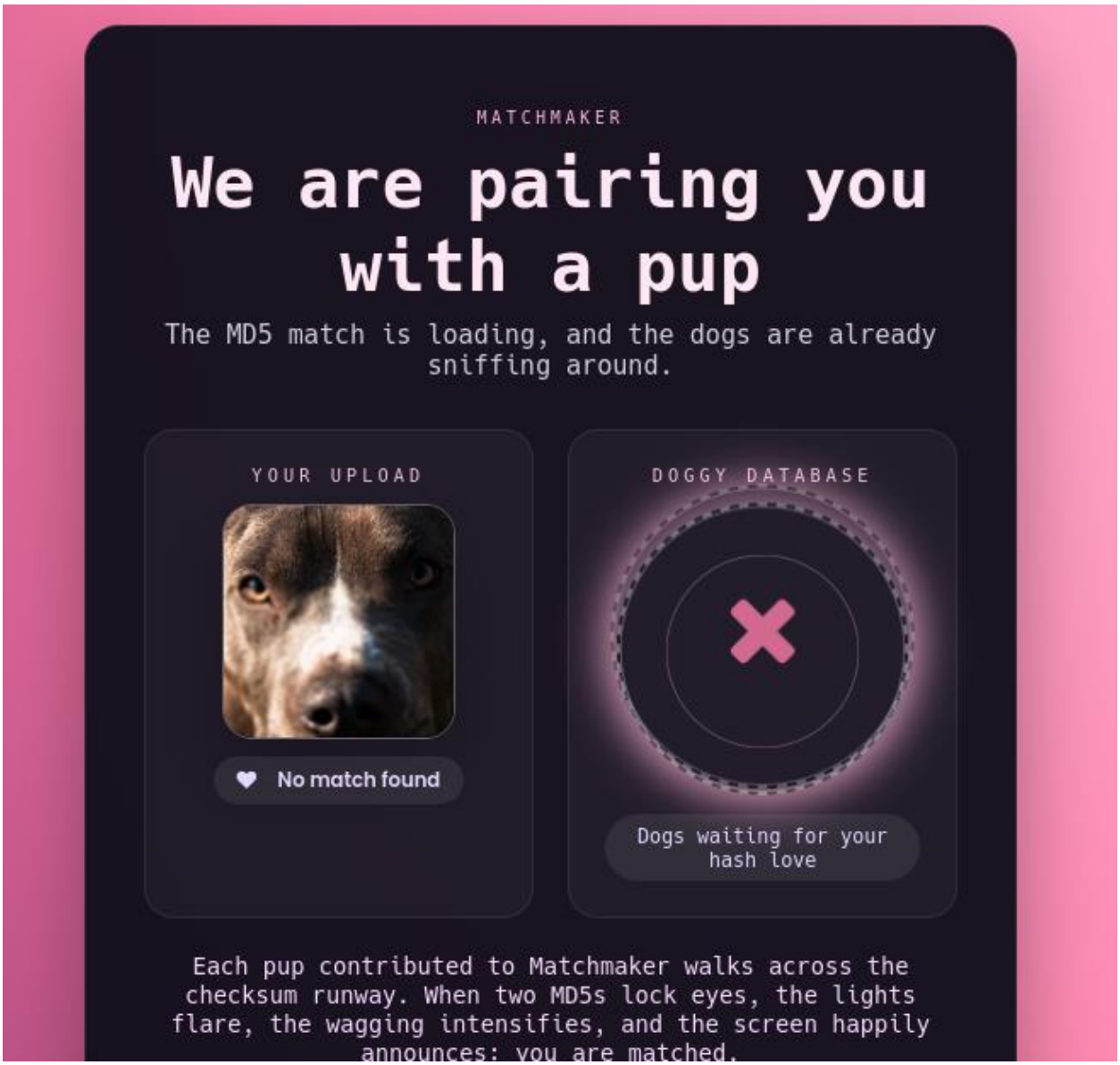
```
┌──(kn999㉿kn999)-[~/Pictures]
└─$ mv md5_data1 dog1.jpg

┌──(kn999㉿kn999)-[~/Pictures]
└─$ mv md5_data2 dog2.jpg

┌──(kn999㉿kn999)-[~/Pictures]
└─$ ls
dog1.jpg  dog2.jpg  thm-ab-bg-1803.png
```

Upload Both Collision Files to the Web App

G back to the web application at http//10.49.147.27. Uploaded the first file (dog1.jpg). The site hashes it and stores it. Now upload the second file (dog2.jpg). Since dog2.jpg has the same MD5 hash as dog1.jpg, the app finds a 'match' — and returns the flag.

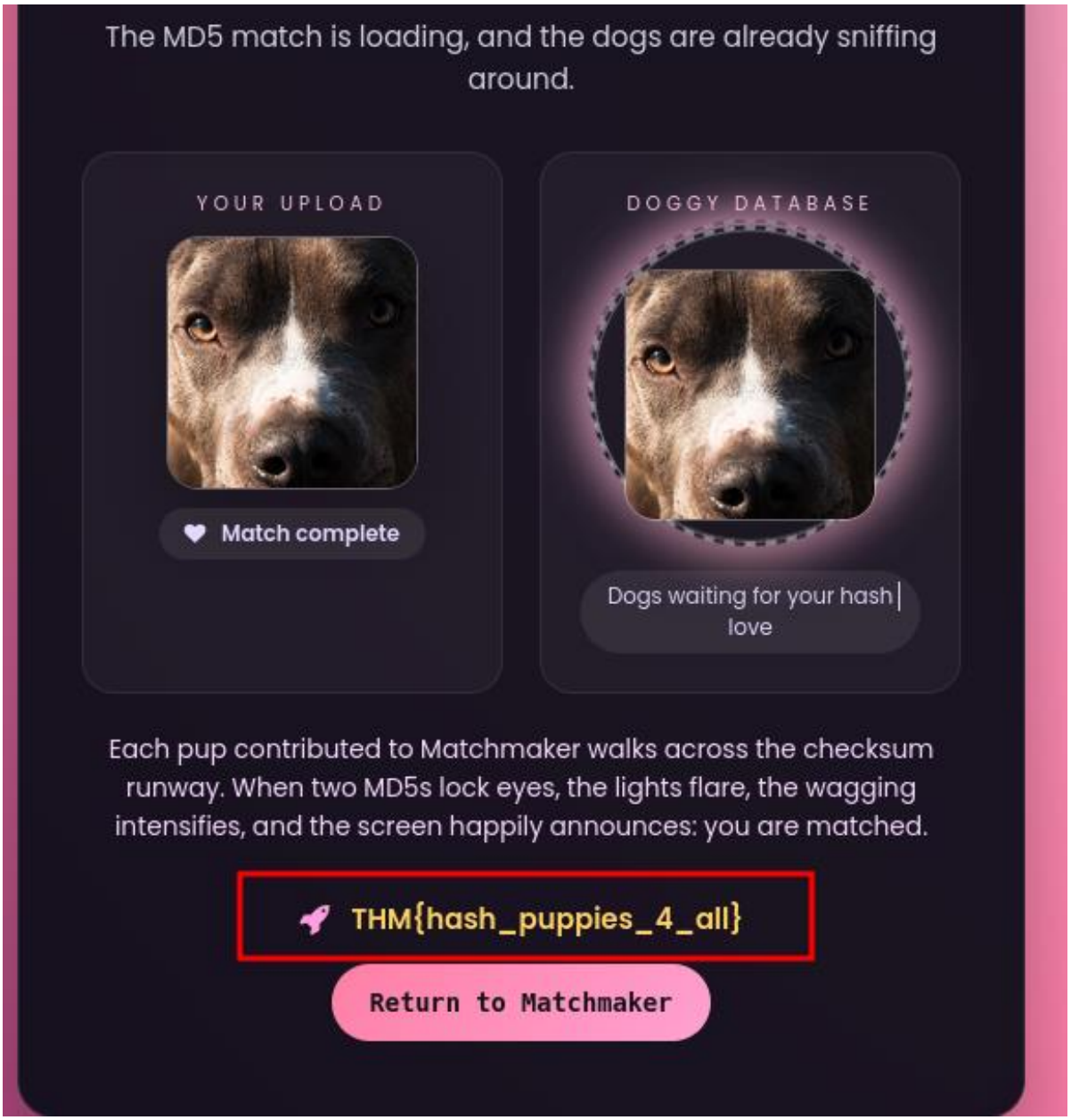
```
# I Uploaded dog1.jpg first via the web form
```

```
# I Then upload dog2.jpg — the app finds its MD5 twin
```



The web app displays the matched dog and reveals the flag:

```
Flag: THM{hash_puppies_4_all}
```



6. Key Concepts Learned

Concept	Key Takeaway
nmap scan	**Revealed ports 22 (SSH) and 80 (HTTP) — guided us to the web app**

MD5 in the app	**The web app uses MD5 hashes to match uploaded images to dog photos**
Hash collision	**Two different files can share the same MD5 hash — MD5 is broken**
FastColl tool	**Generates MD5 collision pairs in seconds using a known attack method**
md5sum verification	**Confirmed both output files had identical 32-character MD5 hashes**
Renaming to .jpg	**The app only accepts image extensions — binary files renamed to .jpg**
Flag obtained	**THM{hash_puppies_4_all} — awarded for successfully forging a match**

7. Summary & Conclusion

This challenge demonstrated a real-world cryptographic attack called an MD5 hash collision. The web application trusted MD5 as a unique identifier for images, but MD5 has been known to be broken since 2004. By using FastColl, we generated two different binary files with an identical MD5 hash and uploaded them to trick the app into declaring a match.

- Ran an nmap scan and found port 80 hosting the dog-matching web app
- Discovered the app uses MD5 hashes to match uploaded images
- Downloaded a real dog image from the site as the FastColl prefix file
- Compiled FastColl and generated two files with the same MD5 hash
- Verified the collision with md5sum, then uploaded both files to get the flag

8. References

- TryHackMe Room — https://tryhackme.com/room/lafb2026e1
- FastColl GitHub — https://github.com/upbit/clone-fastcoll
- FastColl Docker Image — https://hub.docker.com/r/brimstone/fastcoll
- MD5 Collision Research (Lenstra et al., 2005) — https://www.mscs.dal.ca/~selinger/md5collision/
- NIST on MD5 Deprecation — https://csrc.nist.gov